ИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Построение и анализ алгоритмов»

Тема: Расстояние Левенштайна

Вариант: 1

Студент гр. 4343 _____ Чулков В.М.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Вариант

Вар. 1. "Особо заменяемый символ и особо добавляемый символ": цена замены определённого символа отличается от обычной цены замены; цена добавления другого (или того же) определённого символа отличается от обычной цены добавления. Особо заменяемый символ и цена его замены, особо добавляемый символ и цена его добавления — дополнительные входные данные.

Задание 1

Над строкой *ε* (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1.*replace(ε, a, b)* – заменить символ *a* на символ *b*.

2.*insert(ε, a)* – вставить в строку символ *a* (на любую позицию).

3.*delete(ε, b)* – удалить из строки символ *b*.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки *A* и *B*, а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки *A* в строку *B*.

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – *A*; третья строка – *B*.

Выходные данные: одно число – минимальная стоимость операций.

Задание 2

Над строкой *ε* (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1.*replace(ε, a, b)* – заменить символ *a* на символ *b*.

2.*insert(ε, a)* – вставить в строку символ *a* (на любую позицию).

3.*delete(ε, b)* – удалить из строки символ *b*.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки *A* и *B*, а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки *A* в строку *B*.

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – *A*; третья строка – *B*.

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

Задание 3

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример

Для строк pedestal и stien расстояние Левенштейна равно 7:

Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.

Затем необходимо заменить два последних символа: stal -> stie.

Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. (S, $1 \le |S| \le 2550$).

Вторая строка входных данных содержит строку из строчных латинских букв. (T, $1 \le |T| \le 2550$).

Параметры выходных данных:

Одно число L, равное расстоянию Левенштейна между строками S и T.

Описание работы алгоритма

1. Задача нахождения минимальной стоимости преобразования строк (Вариант 1)

Для решения задачи вычисления минимальной стоимости редактирования строк реализован точный алгоритм на основе метода динамического программирования.

Динамическое программирование (построение матрицы)

Основная идея заключается в последовательном заполнении таблицы стоимостей преобразования префиксов одной строки в префиксы другой, с учетом индивидуальных цен для заданных “особых” символов:

Представление состояний:

Процесс преобразования описывается двумерной матрицей (или двумя одномерными массивами для экономии памяти), где каждая ячейка dp[i][j] хранит глобально минимальную стоимость превращения подстроки длины i первой строки в подстроку длины j второй строки.

Учет вариативности цен:

При расчете каждого перехода алгоритм динамически проверяет, совпадает ли текущий обрабатываемый символ с заданными “особо заменяемыми” или “особо добавляемыми” символами.

Если совпадает, применяется особая стоимость, иначе — базовая.

Оценка стоимости (Правило треугольника):

На каждом шаге алгоритм выбирает строго минимальный из трех вариантов: замена символа (переход по диагонали), вставка (переход влево) или удаление (переход вверх), суммируя стоимость прошлого состояния с ценой текущей операции.

Формирование результата:

Минимальная итоговая стоимость всех необходимых операций планомерно накапливается и возвращается из последней вычисленной ячейки (правого нижнего угла матрицы).

2. Задача поиска редакционного предписания и выравнивания строк

Для решения задачи, требующей не только вычисления стоимости, но и вывода последовательности операций (редакционного предписания) с выравниванием строк, реализован алгоритм динамического программирования в связке с обратным ходом (бэктрекингом).

Метод состоит из двух этапов: полного построения матрицы стоимостей всех подзадач и последующего прохода по ней в обратном направлении для пошагового восстановления кратчайшего пути преобразования:

Построение матрицы (Прямой ход): Алгоритм целиком заполняет матрицу состояний размера (N+1) * (M+1), строго фиксируя минимальные цены для замены, вставки и удаления на каждом шаге обхода символов слева направо.

Восстановление маршрута:

После заполнения алгоритм стартует из правого нижнего угла матрицы и двигается к стартовой ячейке. На каждом шаге он проверяет, откуда был совершен оптимальный переход, вычитая из текущей ячейки локальные цены операций для поиска предыдущего состояния.

Приоритет операций:

Для обеспечения логичного выравнивания, при равных стоимостях переходов из разных ячеек, алгоритм отдает строгий приоритет операциям в порядке: совпадение (M) -> замена (R) -> удаление (D) -> вставка (I).

Выравнивание строк:

Параллельно с определением типа операции, алгоритм собирает итоговые массивы выровненных строк, вставляя пробелы-заглушки на местах удалений (в результирующую строку) или на местах вставок (в исходную строку).

3. Задача быстрого вычисления расстояния Левенштейна для длинных строк

Для решения задачи поиска классического расстояния Левенштейна между строками экстремальной длины (до 2550 символов) в условиях жестких ограничений по времени реализован ультра-оптимизированный алгоритм динамического программирования.

Основная идея заключается в отказе от хранения полной матрицы состояний в пользу одного одномерного массива (снижение памяти до O(M)$ и тотальной минимизации накладных расходов внутри вложенных циклов:

Предвычисление данных:

Чтобы исключить условные проверки внутри миллионов итераций двойного цикла, стоимости вставки для всех символов второй строки вычисляются и сохраняются в отдельный кэш-массив до начала основного обхода.

Изоляция состояний (1D DP):

Алгоритм хранит только один актуальный ряд матрицы dp, непрерывно перезаписывая его на лету. Значение диагонального элемента, необходимое для вычисления стоимости замены, сохраняется в отдельной переменной prev_diag перед каждой перезаписью.

Минимизация ветвлений:

Внутри основного цикла полностью исключены вызовы медленных встроенных функций (например, min()). Вместо них используются быстрые тернарные операторы для мгновенного поиска минимальной стоимости перехода из трех возможных.

Скоростной перебор:

Итерация по символам строк выполняется с помощью функции enumerate(), перебирающей элементы на уровне языка Си, что радикально ускоряет доступ к данным по сравнению со стандартным обращением по индексам.

Описание рекурсивной функции

Сигнатура: calculate_min_cost(...)

Назначение: Вычисляет минимальную стоимость преобразования str_a в str_b методом динамического программирования с поддержкой индивидуальных цен для заданных “особых” символов.

Аргументы:

price_rep, price_ins, price_del - базовые стоимости операций замены, вставки и удаления.

str_a, str_b - исходная и получаемая строки.

special_char_rep, special_price_rep - особый символ замены и его цена.

special_char_ins, special_price_ins - особый символ вставки и его цена.

Возвращаемое значение: Целое число - глобально минимальная итоговая стоимость всех операций.

Принцип работы: Инициализируется матрица dp размера (n+1) x (m+1) и заполняются базовые случаи (стоимости полных удалений и вставок). Затем в двойном цикле матрица пошагово заполняется: для каждой ячейки выбирается строго минимальный вариант из трех переходов (замена, вставка, удаление) с учетом базовых или особых цен. Итоговый ответ берется из последней ячейки dp[n][m].

Сигнатура: calculate_and_print_prescription(...)

Назначение: Строит матрицу редакционного расстояния (с учетом особых цен) и выводит оптимальную последовательность операций преобразования строк с их визуальным выравниванием.

Аргументы:

price_rep, price_ins, price_del — базовые цены операций замены, вставки, удаления.

str_a, str_b — преобразуемые строки.

special_char_rep, special_price_rep — особый заменяемый символ и его цена.

special_char_ins, special_price_ins — особый добавляемый символ и его цена.

Возвращаемое значение: Отсутствует (результат в виде трех выровненных строчек печатается напрямую в консоль).

Принцип работы: Сначала заполняется матрица dp, где каждая ячейка хранит минимальную стоимость с учетом базовых и особых цен операций. После заполнения матрицы алгоритм запускает бэктрекинг (обратный ход) из правого нижнего угла к старту. На каждом шаге он выясняет, какое действие привело к текущей стоимости (совпадение, замена, удаление или вставка), и записывает соответствующую букву операции (M, R, D, I) и символы строк, подставляя пробелы в местах разрывов. В финале собранные с конца данные переворачиваются и выводятся на экран.

Сигнатура: get_levenshtein_distance(...)

Назначение: Вычисляет расстояние Левенштейна между двумя строками (с возможностью применения особых цен для специфических символов) с использованием оптимизированного по памяти алгоритма динамического программирования.

Аргументы:

str_a, str_b — исходная и целевая строки.

special_char_rep, special_price_rep — особый символ замены и его цена (опционально).

special_char_ins, special_price_ins — особый символ вставки и его цена (опционально).

Возвращаемое значение: Целое число — минимальная итоговая стоимость всех необходимых операций преобразования.

Принцип работы: Для радикальной экономии памяти (снижение пространственной сложности до O(M)) алгоритм не строит двумерную матрицу, а использует только два одномерных массива: предыдущий (prev_row) и текущий (curr_row) ряды вычислений. Базовые стоимости всех операций (замена, вставка, удаление) жестко равны 1.

Сначала инициализируется prev_row стоимостями базовых или особых вставок. Затем в двойном цикле (по символам первой и второй строки) рассчитывается curr_row. Для каждой ячейки выбирается минимальная цена из трех возможных переходов: по диагонали (замена), слева (вставка) или сверху (удаление). При этом алгоритм проверяет совпадение текущих букв с переданными особыми символами, чтобы применить к ним нестандартные цены вместо базовой единицы. В конце каждой итерации curr_row становится prev_row для следующего шага, а финальный ответ возвращается из последнего элемента сохраненного ряда.

Оценка сложности алгоритма

Динамическое программирование (Поиск минимальной стоимости, Вариант 1)

Временная сложность: Алгоритм вычисляет стоимость преобразования одной строки в другую, заполняя двумерную матрицу. Основная работа происходит в двух вложенных циклах: внешний цикл проходится по символам первой строки (длина n), внутренний — по символам второй (длина m). Внутри цикла выполняется константное количество операций (проверка особых символов и выбор минимума из трех возможных переходов). Время работы предсказуемо и строго пропорционально произведению длин строк.

Итог: временная сложность - O(n * m).

Пространственная сложность: Основная память расходуется на создание и хранение полной двумерной матрицы состояний dp размером (n+1) на (m+1). Дополнительные затраты на переменные минимальны.

Итог: пространственная сложность - O(n * m).

Динамическое программирование с бэктрекингом (Редакционное предписание)

Временная сложность: Алгоритм состоит из двух последовательных этапов. Сначала происходит полное заполнение матрицы стоимостей, что занимает время O(n * m). После этого выполняется обратный ход (бэктрекинг) от правого нижнего угла к левому верхнему для восстановления оптимального пути. Этот обратный проход совершает не более n + m шагов. Поскольку асимптотически O(n * m) растет быстрее, чем O(n + m), этап создания матрицы является доминирующим.

Итог: временная сложность - O(n * m).

Пространственная сложность: Для того чтобы восстановить маршрут, алгоритму абсолютно необходимо держать в оперативной памяти всю

матрицу состояний целиком — это (n+1) * (m+1) ячеек. Дополнительная память выделяется под списки для формирования выровненных итоговых строк, размер которых не превышает n + m.

Итог: пространственная сложность - O(n * m).

Динамическое программирование (Быстрое расстояние Левенштейна)

Временная сложность: Несмотря на радикальное сокращение накладных расходов за счет отказа от медленных встроенных функций Python, предвычисления цен вставок и использования тернарных операторов, структура алгоритма сохраняет два вложенных цикла. Ему по-прежнему необходимо обработать каждую комбинацию символов исходной и целевой строк.

Итог: асимптотическая временная сложность - O(n * m).

Пространственная сложность: Главное преимущество данного метода — отказ от хранения полной матрицы. Память расходуется лишь на два коротких одномерных массива (prev_row и curr_row) размером m+1, а также на один массив кэшированных цен вставки длиной m. Старые данные немедленно удаляются из памяти, что позволяет обрабатывать строки экстремальной длины без риска переполнения.

Итог: пространственная сложность - O(m).

Вывод

В ходе лабораторной работы успешно реализованы три варианта алгоритмов на основе динамического программирования для решения задач поиска редакционного расстояния между строками.

Классический подход и алгоритм с бэктрекингом гарантируют точное нахождение минимальной стоимости и оптимальной последовательности операций (предписания) за O(n * m) по времени и памяти. Однако из-за высоких требований к оперативной памяти они применимы только для строк умеренной длины.

Оптимизированный метод расчета расстояния Левенштейна работает с той же временной сложностью O(n * m), но благодаря переходу на одномерные массивы радикально снижает потребление памяти до O(m). Это позволяет алгоритму успешно обрабатывать объемные тексты (до 2500+ символов) без переполнения памяти и превышения лимитов времени.

Тестирование подтвердило корректность работы всех реализаций: алгоритмы безошибочно находят цены преобразований с учетом особых символов и строят логичные выравнивания строк. Поставленная задача выполнена полностью, а программы могут применяться на практике в зависимости от приоритетов ресурсоемкости (доступной памяти) и необходимости визуализации маршрута.

Приложение

Код программы task1.py:

```
def calculate_min_cost(price_rep: int, price_ins: int, price_del: int,
                       str_a: str, str_b: str,
                                          special_char_rep:  str  =  None,
special_price_rep: int = None,
                                          special_char_ins:  str  =  None,
special_price_ins: int = None) -> int:

    n = len(str_a)
    m = len(str_b)

    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        dp[i][0] = dp[i-1][0] + price_del

    for j in range(1, m + 1):
             if  (special_char_ins  is  not  None)  and  (str_b[j-1]  ==
special_char_ins):
            current_ins_price = special_price_ins
        else:
            current_ins_price = price_ins

        dp[0][j] = dp[0][j-1] + current_ins_price

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if str_a[i-1] == str_b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                  if (special_char_rep is not None) and (str_a[i-1] ==
special_char_rep):
                    current_rep_price = special_price_rep
                else:
                    current_rep_price = price_rep

                  if (special_char_ins is not None) and (str_b[j-1] ==
special_char_ins):
                    current_ins_price = special_price_ins
                else:
                    current_ins_price = price_ins

                dp[i][j] = min(
                    dp[i-1][j-1] + current_rep_price,
                    dp[i][j-1] + current_ins_price,
                    dp[i-1][j] + price_del
                )
           print(f"Матрица после обработки символа '{str_a[i-1]}' (Шаг
{i}):")
        print("     "," ".join([i for i in str_b]))
        c=0
        for row in dp:
            if c!=0:
```

```
                print(str_a[c-1],row)
            else:
                print(" ",row)
            c+=1
        print()

    return dp[n][m]

if __name__ == "__main__":
    prices = input().split()
          p_rep,   p_ins,   p_del   =   int(prices[0]),   int(prices[1]),
int(prices[2])

    s_a = input().strip()
    s_b = input().strip()

    # Флаг: 1 - есть доп. данные, 0 - нет доп. данных
    flag = 0

    sp_char_rep, sp_price_rep = None, None
    sp_char_ins, sp_price_ins = None, None

    if flag == 1:
        extra_rep = input().split()
        sp_char_rep = extra_rep[0]
           sp_price_rep = int(extra_rep[1] if len(extra_rep) > 1 else
input())

        extra_ins = input().split()
        sp_char_ins = extra_ins[0]
           sp_price_ins = int(extra_ins[1] if len(extra_ins) > 1 else
input())

    result = calculate_min_cost(
        price_rep=p_rep, price_ins=p_ins, price_del=p_del,
        str_a=s_a, str_b=s_b,
        special_char_rep=sp_char_rep, special_price_rep=sp_price_rep,
        special_char_ins=sp_char_ins, special_price_ins=sp_price_ins
    )

    print(result)
```

Код программы task2.py:

```
import pprint
def  calculate_and_print_prescription(price_rep:  int,  price_ins:  int,
price_del: int,
                                      str_a: str, str_b: str,
                                          special_char_rep: str = None,
special_price_rep: int = None,
                                          special_char_ins: str = None,
special_price_ins: int = None):

    n = len(str_a)
    m = len(str_b)
```

```
    dp = [[0] * (m + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        dp[i][0] = dp[i-1][0] + price_del

    for j in range(1, m + 1):
              if (special_char_ins is not None) and (str_b[j-1] ==
special_char_ins):
            current_ins_price = special_price_ins
        else:
            current_ins_price = price_ins

        dp[0][j] = dp[0][j-1] + current_ins_price

    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if str_a[i-1] == str_b[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                  if (special_char_rep is not None) and (str_a[i-1] ==
special_char_rep):
                    current_rep_price = special_price_rep
                else:
                    current_rep_price = price_rep

                  if (special_char_ins is not None) and (str_b[j-1] ==
special_char_ins):
                    current_ins_price = special_price_ins
                else:
                    current_ins_price = price_ins

                dp[i][j] = min(
                    dp[i-1][j-1] + current_rep_price,
                    dp[i][j-1] + current_ins_price,
                    dp[i-1][j] + price_del
                )
           print(f"Матрица после обработки символа '{str_a[i-1]}' (Шаг
{i}):")
        print("     "," ".join([i for i in str_b]))
        c=0
        for row in dp:
            if c!=0:
                print(str_a[c-1],row)
            else:
                print(" ",row)
            c+=1
        print()

    i, j = n, m
    ops = []
    res_a = []
    res_b = []

    while i > 0 or j > 0:
         if (special_char_rep is not None and i > 0 and str_a[i-1] ==
special_char_rep):
```

```
            current_rep_price = special_price_rep
        else:
            current_rep_price = price_rep

         if (special_char_ins is not None and j > 0 and str_b[j-1] ==
special_char_ins):
            current_ins_price = special_price_ins
        else:
            current_ins_price = price_ins

         if i > 0 and j > 0 and str_a[i-1] == str_b[j-1] and dp[i][j]
== dp[i-1][j-1]:
                        print(f"[{i}][{j}]  ->  [{i-1}][{j-1}]:  Символы
'{str_a[i-1]}' и '{str_b[j-1]}' совпадают. Операция 'M'.")
            ops.append('M')
            res_a.append(str_a[i-1])
            res_b.append(str_b[j-1])
            i -= 1
            j -= 1

        elif i > 0 and j > 0 and str_a[i-1] != str_b[j-1] and dp[i][j]
== dp[i-1][j-1] + current_rep_price:
                        print(f"[{i}][{j}]  ->  [{i-1}][{j-1}]:  Замена
'{str_a[i-1]}' на '{str_b[j-1]}'. Операция 'R'.")
            ops.append('R')
            res_a.append(str_a[i-1])
            res_b.append(str_b[j-1])
            i -= 1
            j -= 1

        elif i > 0 and dp[i][j] == dp[i-1][j] + price_del:
                        print(f"[{i}][{j}]  ->  [{i-1}][{j}]:  Удаление
'{str_a[i-1]}'. Операция 'D'.")
            ops.append('D')
            res_a.append(str_a[i-1])
            res_b.append(' ')
            i -= 1

        elif j > 0 and dp[i][j] == dp[i][j-1] + current_ins_price:
                         print(f"[{i}][{j}]  ->  [{i}][{j-1}]:  Вставка
'{str_b[j-1]}'. Операция 'I'.")
            ops.append('I')
            res_a.append(' ')
            res_b.append(str_b[j-1])
            j -= 1

    print(" ".join(reversed(ops)))
    print(" ".join(reversed(res_a)))
    print(" ".join(reversed(res_b)))


if __name__ == "__main__":
    prices = input().split()
          p_rep,  p_ins,  p_del  =  int(prices[0]),  int(prices[1]),
int(prices[2])
```

```
    s_a = input().strip()
    s_b = input().strip()

      # Флаг: 1 - есть доп. данные, 0 - нет доп. данных (четвертая
строка)
    flag = 0

    sp_char_rep, sp_price_rep = None, None
    sp_char_ins, sp_price_ins = None, None

    if flag == 1:
        extra_rep = input().split()
        sp_char_rep = extra_rep[0]
           sp_price_rep = int(extra_rep[1] if len(extra_rep) > 1 else
input())

        extra_ins = input().split()
        sp_char_ins = extra_ins[0]
           sp_price_ins = int(extra_ins[1] if len(extra_ins) > 1 else
input())

    calculate_and_print_prescription(
        price_rep=p_rep, price_ins=p_ins, price_del=p_del,
        str_a=s_a, str_b=s_b,
        special_char_rep=sp_char_rep, special_price_rep=sp_price_rep,
        special_char_ins=sp_char_ins, special_price_ins=sp_price_ins
    )
```

Код программы task3.py:

```
def get_levenshtein_distance(str_a: str, str_b: str,
                                          special_char_rep: str = None,
special_price_rep: int = None,
                                          special_char_ins: str = None,
special_price_ins: int = None) -> int:

    n = len(str_a)
    m = len(str_b)

    price_rep = 1
    price_ins = 1
    price_del = 1

    if n == 0:
        cost = 0
        for char in str_b:
                      if (special_char_ins is not None and char ==
special_char_ins):
                cost += special_price_ins
            else:
                cost += price_ins
        return cost

    prev_row = [0] * (m + 1)
    for j in range(1, m + 1):
```

```
            if (special_char_ins is not None) and (str_b[j-1] ==
special_char_ins):
            current_ins_price = special_price_ins
        else:
            current_ins_price = price_ins
        prev_row[j] = prev_row[j-1] + current_ins_price

    for i in range(1, n + 1):
        curr_row = [0] * (m + 1)
        curr_row[0] = prev_row[0] + price_del

        char_a = str_a[i-1]
                if (special_char_rep is not None) and (char_a ==
special_char_rep):
            current_rep_price = special_price_rep
        else:
            current_rep_price = price_rep

        for j in range(1, m + 1):
            char_b = str_b[j-1]

            if char_a == char_b:
                curr_row[j] = prev_row[j-1]
            else:
                     if (special_char_ins is not None) and (char_b ==
special_char_ins):
                    current_ins_price = special_price_ins
                else:
                    current_ins_price = price_ins
                curr_row[j] = min(
                    prev_row[j-1] + current_rep_price,
                    curr_row[j-1] + current_ins_price,
                    prev_row[j] + price_del
                )
        print(f"Шаг {i} (Обработка символа '{char_a}'):")
        print(f"{prev_row}")
        print(f"{curr_row}\n\n")
        prev_row = curr_row

    return prev_row[m]

if __name__ == "__main__":
    s_a = input().strip()
    s_b = input().strip()

       # Флаг: 1 - есть доп. данные, 0 - классическое расстояние
Левенштейна

    flag = 0

    sp_char_rep, sp_price_rep = None, None
    sp_char_ins, sp_price_ins = None, None

    if flag == 1:
        extra_rep = input().split()
        sp_char_rep = extra_rep[0]
```

```
            sp_price_rep = int(extra_rep[1] if len(extra_rep) > 1 else
input())

        extra_ins = input().split()
        sp_char_ins = extra_ins[0]
            sp_price_ins = int(extra_ins[1] if len(extra_ins) > 1 else
input())

    print(get_levenshtein_distance(
        s_a, s_b,
        sp_char_rep, sp_price_rep,
        sp_char_ins, sp_price_ins
    ))
```